

# Knowledge Graphs

## Lecture 9: Complexity of SPARQL and Datalog

Markus Krötzsch

Knowledge-Based Systems

TU Dresden, 9 Dec 2025

More recent versions of this slide deck might be available.  
For the most current version of this course, see  
[https://iccl.inf.tu-dresden.de/web/Knowledge\\_Graphs/en](https://iccl.inf.tu-dresden.de/web/Knowledge_Graphs/en)

# Review

## Datalog is a general language for recursive, relational queries

- Easy to adopt to graphs (edges = special relations)
- Plain Datalog is a “pure” paradigm without the technical extensions of real query languages (esp. data types, filters)
- Adding negation is useful, but the interplay with recursion must be limited
- Adding aggregation must consider similar issues
- Datatype support can follow a weak or strong typing approach (RDF-based systems prefer weak)

## Nemo is a free rule engine for knowledge graphs

- Data can be loaded from various sources, including SPARQL endpoints
- Support for datatypes, SPARQL-like functions and filters, stratified negation, and aggregation
- Syntax inspired by RDF and SPARQL to work with IRIs and datatype literals

Web-based Datalog reasoner at <https://tools.iccl.inf.tu-dresden.de/nemo/>

We are still looking for participants.

Book now:

<https://tinyurl.com/nemostudy2025>

60min — APB — 20 EUR compensation

# Complexity of SPARQL

# Finding BGP solutions

How can we compute solutions to BGPs?

## Possible approach:

1. Find solutions to triple patterns
2. Compute joins of partial solutions

By Theorem 6.6,  $\text{BGP}_G(P)$  is the join of the solution multisets of each triple pattern in  $P$ .  
(Blank nodes might need to be replaced by variables that are projected away later.)

How hard is this? (on a graph with  $n$  edges)

1. Can be solved by iterating over all edges:  $O(n)$  (linear)
2. We defined  $\text{Join}(\Omega_1, \Omega_2) = \{\mu_1 \uplus \mu_2 \mid \mu_1 \in \Omega_1, \mu_2 \in \Omega_2, \text{ and } \mu_1 \text{ and } \mu_2 \text{ are compatible}\}$ .  
Therefore  $\text{Join}(\Omega_1, \Omega_2)$  is of size  $O(|\Omega_1| \times |\Omega_2|) \in O(n^2)$  (quadratic)  
But joining results of  $k$  triple patterns is in  $O(n^k)$  (exponential)!

~> worst-case exponential-time query answering algorithm

# Review: Computational complexity

Computational complexity provides tools for estimating

- how hard a problem is
- based on the effort an algorithm needs to solve it

To classify algorithms, we distinguish:

- **computational models**: deterministic, non-deterministic, probabilistic, quantum, ...
- **constrained resources**: time (steps), space (memory), ...
- **resource bounds**: polynomial, exponential, ... (measured wrt. to input size)

Such complexity classifications are rather robust measures of a problem's "difficulty" and do not depend on implementation details.

# Review: Some complexity classes

deterministic

$$P = PTIME = \bigcup_{d \geq 1} DTIME(n^d)$$

polynomial time

$$EXP = EXP TIME = \bigcup_{d \geq 1} DTIME(2^{n^d})$$

exponential time

$$L = LOGSPACE = DSPACE(\log n)$$

logarithmic space

$$PSPACE = \bigcup_{d \geq 1} DSPACE(n^d)$$

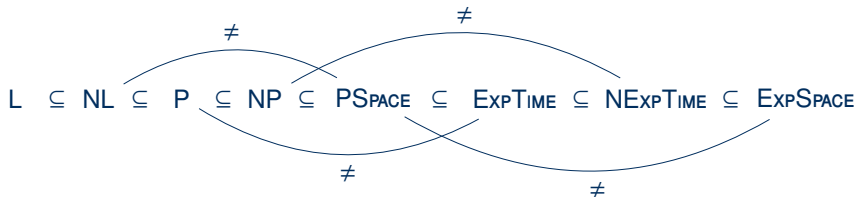
polynomial space

non-deterministic

$$NP = \bigcup_{k \geq 1} NTIME(n^k)$$

$$NEXP = NEXP TIME = \bigcup_{k \geq 1} NTIME(2^{n^k})$$

$$NL = NLOGSPACE = NSPACE(\log n)$$



# Review: The class NP

NP is an extremely common class for challenging problems in practice.  
It can be defined in two ways:

## **Nondeterministic polynomial time**

- Problems in NP can be solved by a non-deterministic algorithm
- In time bounded by a polynomial

## **Polynomial verification**

- All problems in NP have polynomial “solutions”: short certificates that prove all “yes” answers
- The correctness of such certificates can be verified in polynomial time

NP problems are search problems – searching for a right solution among the exponentially many potential solutions – but even the best known algorithms may take exponential time.



# Finding BGP solutions

**Observation:** It is easy to check if a given mapping of bnodes and variables produces a solution:

- Simply verify that the mapped triples are contained in the given graph
- Can be done in quadratic time ( $\# \text{ triples in pattern} \times \# \text{ edges in graph}$ )

In other words: the problem (as a decision problem) is in NP.

It turns out this is the best we can do:

**Theorem 9.1:** Determining if a BGP has solution mappings over a graph is NP-complete (with respect to the size of the pattern).

**Proof:**

- Inclusion: guess mapping for bnodes and variables; check if guess was correct.
- Hardness: by reduction from a known NP-hard problem

# Review: Polynomial many-one reductions

To compare the hardness of problems, we ask which problems can be **reduced** to others.

**Definition 9.2:** A language  $L_1 \subseteq \Sigma^*$  is **polynomially many-one reducible** to  $L_2 \subseteq \Sigma^*$ , denoted  $L_1 \leq_p L_2$ , if there is a polynomial-time computable function  $f$  such that for all  $w \in \Sigma^*$

$$w \in L_1 \quad \text{if and only if} \quad f(w) \in L_2.$$

**Intuition:** If  $L_1 \leq_p L_2$ , then:

- We can solve a problem of  $L_1$ , by reducing it to a problem of  $L_2$
- Therefore  $L_1$  is “at most as difficult” as  $L_2$  (modulo polynomial effort)

**Definition 9.3:** A problem  $C$  is **NP-complete** if  $C \in NP$  and, for every problem  $L \in NP$ , we find  $L \leq_p C$ .

**Intuition:** NP-complete problems are the “hardest” problems in NP since they hold the key to solving all other problems in NP. For a more refined understanding, see course “Complexity Theory”.

# From 3-colourability to BGP matching

The problem of **graph 3-colourability (3Col)** is defined as follows:

**Given:** An undirected graph  $G$

**Question:** Can the vertices of  $G$  be assigned colours red, green and blue so that no two adjacent vertices have the same colour?

It is known that this problem is NP-complete (and in particular NP-hard).

We can find a polynomial many-one reduction from **3Col** to BGP matching:

- A given graph  $G$  is mapped to a BGP  $P_G$  by introducing, for each undirected edge  $e-f$  in  $G$ , two triples  $?e \text{ <edge> } ?f$  and  $?f \text{ <edge> } ?e$ .
- We consider the RDF graph  $C$  given by

`<red> <edge> <green>, <blue> .`  
`<green> <edge> <red>, <blue> .`  
`<blue> <edge> <green>, <red> .`

Then  $P_G$  has a solution mapping over  $C$  if and only if  $G$  is 3-colourable. □

# NP-hardness another way

A typical NP-complete problem is satisfiability of propositional logic formulae:

The problem of **propositional logic satisfiability (SAT)** is defined as follows:

**Given:** An propositional logic formula  $\varphi$

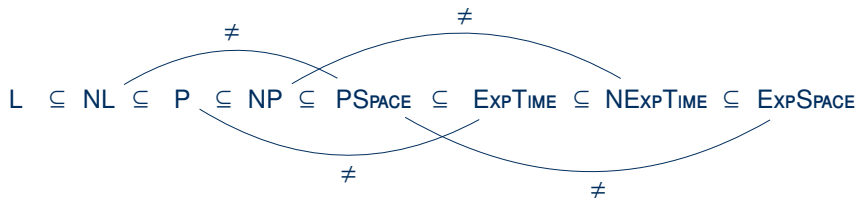
**Question:** Is it possible to assign truth values to propositional variables in  $\varphi$  such that the formula evaluates to true?

**Exercise:** Give a direct reduction from **SAT** to SPARQL query answering, without using BGPs.

This shows (in another way) that SPARQL query answering is NP-hard. However, it is actually harder than that.

# Beyond NP

In complexity theory, space is usually more powerful than time  
(intuition: space can be reused; time, alas, cannot)



Space restrictions can also be used for non-deterministic algorithms, but by [Savitch's Theorem](#), this often does not give additional expressive power:  $PSPACE = NPSPACE$

Completeness again is defined by polynomial reductions:

**Definition 9.4:** A problem  $C$  is  **$PSPACE$ -complete** if  $C \in PSPACE$  and, for every problem  $L \in PSPACE$ , we find  $L \leq_p C$ .

# Quantified Boolean Formulae

A **QBF** is a formula of the following form:

$$Q_1X_1.Q_2X_2.\cdots Q_\ell X_\ell.\varphi[X_1,\dots,X_\ell]$$

where  $Q_i \in \{\exists, \forall\}$  are quantifiers,  $X_i$  are propositional logic variables, and  $\varphi$  is a propositional logic formula with variables  $X_1, \dots, X_\ell$  and constants  $\top$  (true) and  $\perp$  (false)

## Semantics:

- Propositional formulae without variables (only constants  $\top$  and  $\perp$ ) are evaluated as usual
- $\exists X.\varphi[X]$  is true if either  $\varphi[X/\top]$  or  $\varphi[X/\perp]$  are true
- $\forall X.\varphi[X]$  is true if both  $\varphi[X/\top]$  and  $\varphi[X/\perp]$  are true

(where  $\varphi[X/\top]$  is “ $\varphi$  with  $X$  replaced by  $\top$ , and similar for  $\perp$ )

# Hardness of QBF Evaluation

**TRUEQBF** is the following problem:

**Given:** A quantified boolean formula  $\varphi$

**Question:** Does  $\varphi$  evaluate to true?

This is a rather difficult question:

**Example 9.5:** A propositional formula  $\varphi$  with propositions  $p_1, \dots, p_n$  is satisfiable if  $\exists p_1 \dots \exists p_n. \varphi$  is a true QBF, i.e., **SAT** reduces to **TRUEQBF** (so it is NP-hard).

The QBF  $\varphi$  is a tautology if  $\forall p_1 \dots \forall p_n. \varphi$  is a true QBF, i.e., tautology checking reduces to **TRUEQBF** (so it is coNP-hard).

In fact, it is known that **TRUEQBF** is harder than both NP and coNP:

**Theorem 9.6:** **TRUEQBF** is PSPACE-complete.

(without proof; see course “Complexity Theory”)

# Universal quantifiers in SPARQL

To show NP-hardness, we used the fact that SPARQL can naturally express existential quantifiers, since we always ask “does a match for this query exist”?

Can we also express universal quantifiers? — Yes:

**Example 9.7:** In Wikidata, find bands all of whose (known) members are female.

```
SELECT ?band
WHERE {
  ?band wdt:P31 wd:Q215380 . # ?band instance of: band
  ?band wdt:P527 [] . # ?band has part: [] (at least one known member)
  FILTER NOT EXISTS {
    ?band wdt:P527 ?member . # ?band has part: ?member
    FILTER NOT EXISTS {
      ?member wdt:P21 wd:Q6581072 # ?member sex or gender: female
    }
  }
}
```



# SPARQL is PSPACE-hard

The PSPACE-hardness of **TRUEQBF** + the encoding universal quantifiers yield:

**Theorem 9.8:** Deciding whether a SPARQL query has any results is PSPACE-hard, even over an empty RDF graph.

**Proof:** We reduce QBF formulae to SPARQL queries. A QBF

$Q_1X_1.Q_2X_2.\dots Q_\ell X_\ell.\varphi[X_1,\dots,X_\ell]$  is transformed to SPARQL in the following steps:

1. Replace every sub-formula of the form  $\forall X_i.\psi$  by  $\neg\exists X_i.\neg\psi$  (and delete any double  $\neg\neg$ ).
2. Replace the innermost boolean formula ( $\varphi$  or  $\neg\varphi$ ) by an expression **FILTER** ( $\hat{\varphi}$ ) where  $\hat{\varphi}$  is  $(\neg)\varphi$  written using SPARQL Boolean functions &&, ||, and !, and with each propositional variable  $X_i$  replaced by a unique SPARQL variable ?Xi.
3. Replace every sub-expression of the form  $\neg\exists X_i.\psi$  with  
**FILTER NOT EXISTS { VALUES ?Xi {true false}  $\psi$  }**
4. Replace every sub-expression of the form  $\exists X_i.\psi$  with  
**FILTER EXISTS { VALUES ?Xi {true false}  $\psi$  }**

From the resulting SPARQL expression P, create the query:

**SELECT \* WHERE { VALUES ?x {"QBF is true!"} P }**

## SPARQL is PSPACE-hard (2)

It is not hard to see that this transformation works as desired: the resulting query has a solution mapping  $\{x \mapsto \text{"QBF is true!"}\}$  if and only if the QBF is true.

**Example 9.9:** Consider the QBF  $\forall p. \exists q. ((\neg p \wedge q) \vee (p \wedge \neg q))$ . Eliminating  $\forall$  yields  $\neg \exists p. \neg \exists q. ((\neg p \wedge q) \vee (p \wedge \neg q))$ . We then obtain the following SPARQL query:

```
SELECT * WHERE {  
  VALUES ?x {"QBF is true!"}  
  FILTER NOT EXISTS { VALUES ?p {true false}  
    FILTER NOT EXISTS { VALUES ?q {true false}  
      FILTER ( (! ?p && ?q) || (?p && ! ?q) )  
    }  
  }  
}
```

# Is SPARQL practical?

$PSPACE$ -hard problems are highly intractable and hard to implement in practice.

Is SPARQL practically feasible at all?

Apparently yes:

- We have seen implementations
- Other widely used query languages, such as SQL, have similar complexities

Is complexity theory useless?

No, but we should measure more carefully:

- Our proofs (for NP and  $PSPACE$ ) turn hard problems into hard queries
- We hardly need RDF data at all

In practice, databases grow very big, while queries are rather limited!

(Wikidata has billions of triples; typical Wikidata query have less than 100 triple patterns [Malyshev et al., ISWC 2018])

# More fine-grained complexity measures

## Combined Complexity

Input: Query  $Q$  and RDF graph  $G$

Output: Does  $Q$  have answers over  $G$ ?

- ~ estimates complexity in terms of overall input size
- ~ “2KB query/2TB database” = “2TB query/2KB database”
- ~ study worst-case complexity of algorithms for fixed queries:

## Data Complexity

Input: RDF graph  $G$

Output: Does  $Q$  have answers over  $G$ ? (for fixed query  $Q$ )

- ~ we can also fix the database and vary the query:

## Query Complexity

Input: SPARQL query  $Q$

Output: Does  $Q$  have answers over  $G$ ? (for fixed RDF graph  $G$ )

## Below P

Our previous proofs show high query complexity (hence also high combined complexity). For data complexity, we get much lower complexities, starting below polynomial time.

**Definition 9.10:** The class NL of languages decidable in **logarithmic space on a non-deterministic Turing machine** is defined as  $NL = NSPACE(\log(n))$ .

**Note:** When restricting Turing machines to use less than linear space, we need to provide them with a separate read-only input tape that is not counted (since the input of length  $n$  cannot fit into  $\log(n)$  space itself).

**Intuition:** The memory of a logspace-bounded Turing machine (deterministic or not) is just enough for the following:

- Store a fixed number of binary counters (with at most polynomial value)
- Store a fixed number of pointers to positions in the input
- Compare the values of counters and target symbols of pointers

It is known that  $NL \subseteq P \subseteq NP$  (and all inclusions are believed to be strict, though this remains unproven)

# Data complexity of SPARQL

The problem of **directed graph reachability** (also known as s-t-reachability) is defined as follows:

**Given:** A directed graph  $G$  and two vertices  $s$  and  $t$

**Question:** Is there a directed path from  $s$  to  $t$ ?

This can be solved in NL:

- Starting from  $s$ , non-deterministically move to a successor vertex
- Terminate when moving to  $t$  (success) or after making more moves than vertices in the graph (failure)

This runs in logarithmic space: one pointer to current vertex, one counter

Directed graph reachability is furthermore known to be NL-hard, so we get:

**Theorem 9.11:** Deciding if a SPARQL query has any solutions is NL-hard in terms of data complexity.

**Proof:** Directed graph reachability is easily reduced: encode graph in RDF, and use a single property path pattern with  $*$  to check reachability. □

# Upper bounds

**Important note:** All of our results so far were **lower bounds**, showing that SPARQL is **at least as hard** as the given class. We have not shown that SPARQL queries can actually be answered in the given bounds.<sup>1</sup>

## How to obtain upper bounds?

- Give an algorithm
- Show that it can run within the required bounds (with respect to query size and/or data size)

**Problem:** SPARQL has a large number of features that an algorithm would need to consider, making algorithms rather complex and harder to verify

↪ sketch algorithms for basic cases only

---

<sup>1</sup>We have not even shown that SPARQL query answers are computable at all. SQL query answers, e.g., are not, if all SQL features are allowed.

# Answering queries in PSPACE

**Note:** A single query can have exponentially many solutions, so the result does not fit into polynomial space. But a polynomial space algorithm could still discover all solutions (and stream them to an output).

## Algorithm sketch:

- Iterate over all possible variable and bnode bindings, storing them one by one (possible in polynomial space)
- Verify query conditions for the given binding (possible in polynomial space for most features, e.g., triple patterns, property path patterns, filters, union, minus, ...)

## Where this sketch is lacking:

- We should check complexity of all filter conditions and functions
- We did not clarify how to handle subqueries and aggregates
- Result values can become exponentially large (e.g., by repeated string doubling using **BIND**), so a smarter representation of values has to be used



# Answering queries in NL for data

We can use the same approach for worst-case optimal query answering with respect to the size of the RDF graph (data complexity):

## Algorithm sketch:

- Iterate over all possible variable and bnode bindings, storing one at a time
- Verify query conditions for the given binding

↪ If the query is fixed, the bindings can be stored using a fixed number of pointers.

↪ For most operations, it is again clear that they are possible to verify in NL

This includes many numeric aggregates and arithmetic operations.

Again, we omit many details here that would need careful discussion.

**Note:** In terms of the size of the data, values can not be exponentially but merely polynomially large, since the query is constant now; but one still needs to explain how to represent this.

# Complexity of Datalog

# Datalog complexity

## **Fact 9.12:** Datalog query answering is

- $\text{ExpTime}$ -complete in combined and query complexity
- $P$ -complete in data complexity

See course “Database Theory” for details and full proofs; upper bounds are easy to derive from our operational semantics.

## **Discussion:**

- Comparison to SPARQL:  $\text{PSPACE} \subseteq \text{ExpTime}$  and  $\text{NL} \subseteq P$
- However,  $\text{ExpTime}$ -hardness here requires **unbounded growth of predicate arity**
  - $\leadsto$  small predicate arities may simplify reasoning
  - $\leadsto$  for fixed arities, bounds are similar to SPARQL (apply one rule = answer one BGP)
- $P$ -hardness is tied to having multiple IDB predicates per rule body
  - $\leadsto$  with single IDBs per body (**linear Datalog**), we get  $\text{PSPACE}$  (comb.) and  $\text{NL}$  (data)

# Expressive Power

# Expressive power and the limits of SPARQL

The **expressive power** of a query language is described by the question:

“Which sets of RDF graphs can I distinguish using a query of that language?”

## More formally:

- Every query defines a set of RDF graphs: the set of graphs that it returns at least one result for
- However, not every set of RDF graphs corresponds to a query (exercise: why?)

**Note:** Whether a query has any results at all is not what we usually ask for, but it helps us here to create a simpler classification. One could also compare query results over a graph and obtain similar insights overall.

**Definition 9.13:** We say that a query language  $Q_1$  is **more expressive** than another query language  $Q_2$  if it can characterise strictly more sets of graphs.

# Complexity limits expressivity

**Intuition:** The lower the complexity of query answering, the lower its expressivity.

**Question:** which complexity are we talking about here? — data complexity!

- Given a set of RDF graphs that we would like to classify,
- we ask if there is one (fixed) query that accomplishes this.

If classifying the set of graphs encodes a computationally difficult problem, then the query evaluation has to be at least as hard as this problem with respect to data complexity.

**Example 9.14:** We have argued that SPARQL queries can evaluate QBF, and we could encode QBF in RDF graphs (in many reasonable ways). However, there cannot be a SPARQL query that recognises all RDF graphs that encode a true QBF, since this problem is  $PSPACE$ -complete, which is known to be not in NL.

# Complexity is not the same as expressivity

## **Complexity-based arguments are often quite limited:**

- They only apply to significantly harder problems
- Additional assumptions are often needed (e.g., it is assumed that  $NL \neq NP$ , but it was not proven yet)
- Typically, query languages cannot even solve all problems in their own complexity class (i.e., they do not “capture” this class)

~> Direct arguments for non-expressivity need to be sought.

## Example: Complexity $\neq$ expressivity

The problem of **parallel reachability** is defined as follows:

**Given:** An RDF graph  $G$ ; two vertices  $s$  and  $t$ ; and two RDF properties  $p$  and  $q$

**Question:** Is there a directed path from  $s$  to  $t$ , where each two neighbouring nodes on the path are connected by both a  $p$ -edge and a  $q$ -edge?

**Proposition 9.15:** Parallel reachability is in NL.

**Proof:** The check can be done using a similar algorithm as for s-t-reachability, merely checking for two edges in each step. □

**Proposition 9.16:** SPARQL cannot express parallel reachability.

**Proof:** The only SPARQL feature that can check for paths are property path patterns, but:

- a match to a property path pattern is always possible using only vertices of degree 2 on the path; higher degrees can only be enforced for a limited number of nodes that are matched to query variables
- the query requires an arbitrary number of nodes of degree 4 on the path □



# Other structural limits to SPARQL expressivity

SPARQL's regular recursions is also limited in many other cases:

- Non-regular path languages cannot be expressed
- “Wide” paths consisting of repeated graph patterns cannot be expressed
- Tree-like patterns and other non-linear patterns cannot be expressed
- “Nested regular expressions” with tests cannot be expressed

# Limits by design

Besides mere expressivity, SPARQL also has some fundamental limits since it simply has no support for some query or analysis tasks:

- SPARQL is lacking **some datatypes** and matching filter conditions, most notably geographic coordinates (major RDF databases add this)
- SPARQL cannot talk about **path lengths**, e.g., one cannot retrieve the length of the shortest connecting path between two elements
- SPARQL cannot **return paths** (of a priori unknown length) in results
- SPARQL has no support for **recursive/iterative computation**, e.g., for Page Rank or other graph algorithms

**Potential reasons:** **performance concerns** (e.g., Page Rank computation would mostly take too long; longest path detection is NP-complete [in data complexity!]), **historic coincidence** (geo coordinates not in XML Schema datatypes); **design issues** (handling paths in query results would require many different constructs)

# Datalog Expressivity

How expressive is Datalog?

**Definition 9.17:** The expressive power of a Datalog-like language is given by the set of functions from input databases to output databases that one can express with such programs.

Since data complexity is P-complete, every Datalog-expressible function is computable in polynomial time.

Are all polytime computable functions between databases expressible in Datalog?

No!

# Expressive Limits of Datalog

Without negation, Datalog is monotonic, but many polytime functions are not.

## Examples:

- Any program that requires input negation
- Parity: derive “yes()” if there is an even number of constants  
     $\leadsto$  Datalog cannot count (not even with stratified negation)

**Capturing PTime:** To capture all polytime functions, two features must be added:

- input negation (stratified negation only on EDB atoms)
- successor order: predicates `first`, `succ`, and `last` that define an (arbitrary) total order on the domain

However, programs that rely on `succ` are usually not practical, and not really “logical” at all. See course “Database Theory” for further details.

# Summary

SPARQL is PSPACE-complete for query and combined complexity<sup>1</sup>

SPARQL is NL-complete for data complexity, hence practically tractable and well parallelisable<sup>1</sup>

Datalog is EXPTIME-complete for query and combined complexity, and P-complete for data complexity

Expressivity is limited by data complexity, but often smaller (e.g., for SPARQL and Datalog)

## What's next?

- Ontologies
- Constraint languages for knowledged graphs

---

<sup>1</sup>The matching upper bound has not been proven with the full set of features.